# PDF 2 — Engineering & Operations Lessons (VPR-Space reproduction)

*Cập nhật 07/11/2026. Ghi lại kinh nghiệm thực tế: cấu hình môi trường, vận hành Kaggle, các lỗi đã gặp và cách sửa.*

## 1. Kaggle: những điều đã kiểm chứng

| Hạng mục | Thực tế đo được |
|---|---|
| Phiên L4 | Có qua cuộc thi `gemma-4-developer-agent` (id 149921). Metadata: `machine_shape: NvidiaL4`, `competition_sources: [gemma-4-developer-agent]`, Internet **tắt**. Vào thẳng RUNNING, không phải xếp hàng |
| Phần cứng 1 phiên L4 | **4× NVIDIA L4 (23 GB)**, driver 580.178, **46 vCPU**, **188 GB RAM**, `/dev/shm` 85 GB, `/tmp` ~966 GB trống, `/kaggle/working` 19,5 GiB |
| Phần mềm mặc định | Python **3.13.16**, torch 2.11.0+cu130, torchvision 0.26, numpy 2.1.3, Pillow 13.0, timm 1.0.30, pytorch_lightning 2.6.6, kornia 0.9.0, **không có xformers** |
| Quota | `kaggle quota` hiện giờ đã dùng và còn lại; làm mới 00:00 UTC thứ Bảy. Tài khoản có thể dùng vượt 30 h nếu phiên bắt đầu trước khi hết quota |
| Log | `kaggle kernels logs` (kể cả `-f`) **không trả log khi phiên đang chạy**. Vì vậy quy tắc "không có đầu ra sau 8/10 phút" phải dựa vào **trạng thái hàng đợi** |
| Output | `/kaggle/working` tối đa ~20 GB; nên đóng gói ít file. Notebook CPU có Internet tải từ GitHub release khoảng **100–150 MB/s nếu tải song song nhiều phần**, nhưng chỉ ~12 MB/s nếu tải tuần tự một luồng (19 GB mất 27 phút) |
| Chia sẻ dữ liệu | Output của notebook **riêng tư** chỉ gắn được vào notebook cùng tài khoản (`kernel_sources`). Vì vậy mỗi tài khoản GPU tự chạy notebook chuẩn bị dữ liệu và deps (CPU, không tốn quota GPU) |

## 2. Môi trường ghim của tác giả, chạy offline

- Tải bằng `pip download --only-binary=:all: -r requirements_pinned.txt --extra-index-url https://download.pytorch.org/whl/cu124`, với Python 3.13. Có đủ wheel cp313 cho `torch 2.6.0+cu124` và các gói ghim.
- Venv: `python -m venv` **lỗi ensurepip** trên Kaggle. Cách xử lý: `python -m venv --without-pip` rồi `python -m pip --python <venv>/bin/python install --no-index --find-links wheels -r ...`.
- Gói không có trong file freeze của tác giả nhưng code upstream cần ngay lúc import hoặc khởi tạo model. Cài bằng `--no-deps`:
  - `pytorch_lightning`, `lightning_utilities`, `torchmetrics` (SALAD import),
  - **`pytorch_metric_learning`** (SALAD khởi tạo *loss huấn luyện* ngay trong `VPRModel.__init__`),
  - `fast_pytorch_kmeans` → **`psutil`** (AnyLoc),
  - `timm` (DINOv3).

- **Bài học lớn**: `pip install X` *có kèm dependencies* trong môi trường mặc định của Kaggle sẽ **tự nâng cấp torch** và làm hỏng torchvision (`operator torchvision::nms does not exist`). Luôn dùng `--no-deps` khi cài thêm vào môi trường hệ thống.
- **Luôn tự kiểm tra bộ wheel**: cài vào venv sạch rồi import thử *ngay trong notebook deps*. Bước này đã bắt được 4 lỗi trước khi tốn quota GPU.

## 3. torch.hub và Hugging Face offline

- Gọi `torch.hub.load("owner/repo", ...)` sẽ dò `github.com/.../tree/main` để biết nhánh mặc định. Sau proxy thì lỗi trả về là HTTP 400, và torch ném ngoại lệ thay vì dùng cache. Trên Kaggle không có Internet thì lỗi là URLError và torch tự rơi về cache. → Ghi đè `torch.hub._parse_repo_info` để luôn trả về thư mục cache đã đặt sẵn (repo ghim theo commit). Code và trọng số không đổi, chỉ bỏ bước dò mạng.
- Tên thư mục cache phải khớp **chính xác** chuỗi gọi trong code: `amaralibey_bag-of-queries_main` (chữ thường), `gmberton_cosplace_main`, `AnyLoc_DINO_main`... Tất cả repo hub đều dùng nhánh `main`.
- HF: dùng `HF_HOME` (copy bằng `cp -rs`) cùng `HF_HUB_OFFLINE=1`. MegaLoc tải `config.json` trong khối try/except, nên offline vẫn chạy.

## 4. Kiểm thử trước khi chạy GPU

Đã thực hiện đúng quy trình "rà soát rồi mới chạy":

1. Chạy khô `metrics_all.py` trên dữ liệu giả có GT và pose thật (16/16 cặp).
2. Chạy đường MegaLoc của tác giả **offline** trên CPU.
3. Chạy **toàn bộ bootstrap** trên một cây `/kaggle/input` giả (11 phần tar giả, cache giả).

Các lỗi bắt được trước khi tốn GPU:

- Worker ghi file timing khi thư mục output chưa tồn tại.
- torch.hub dò mạng (HTTP 400).
- `None` trong bảng Fig. 6.
- Cân bằng tải GPU phải tính theo pixel: ảnh MADMAX 2064×1544 nặng gấp 5–8 lần ảnh SPICE.

## 5. Tối ưu tốc độ

- Suy luận chia theo **(cấu hình, trajectory)** trên 4 GPU bằng thuật toán LPT. DDP không cần, vì không có gradient. Batch con tự giảm khi OOM.
- Hai vocabulary của AnyLoc (global và unstructured) dùng **chung một lượt ViT-G**, tiết kiệm 50% phần tốn kém nhất.
- DenseVLAD: SIFT của OpenCV mất **50 s/ảnh** (~120 CPU-giờ). Bản kornia trên GPU nhanh (~1 s/ảnh) nhưng **không tương đương VLFeat** (xem §8). Bản cuối gọi chính VLFeat 0.9.21 qua ctypes, chạy song song trên 44/46 vCPU của phiên L4 (mỗi process 1 luồng BLAS). Tốc độ 1,2 s/ảnh 640×480 mỗi lõi, nên ~30 phút cho cả bộ.

- Giải nén 19 GB dữ liệu chạy song song với lúc dựng venv.

## 6. Bảo mật

- Token Kaggle chỉ nằm trong `/root/.kaggle/accounts.json` (quyền 600). Wrapper `kg` đọc token theo tên tài khoản và không bao giờ in ra.
- Trước mỗi commit đều quét chuỗi `KGAT_`.

## 7. Thu và ghép kết quả

- Phiên lỗi ở một trajectory (AnyLoc MadMax-E0, wave 2) được **vá bằng một phiên riêng**, chỉ chạy trajectory đó. Sau đó ghép metric theo cặp bằng `kaggle/merge_results.py`.
  - Ghép như vậy hợp lệ vì metric của mỗi cặp chỉ phụ thuộc descriptor DB và Q của chính cặp đó; descriptor tính độc lập từng ảnh.
  - Cặp xuất hiện ở hai nguồn phải trùng nhau đến 1e-9, nếu không công cụ dừng.
- Phiên chia theo trajectory (`b-anyloc-demo-p1/p2`): tải descriptor (`kernels output --file-pattern`) rồi chạy `metrics_all.py` cục bộ (`kaggle/merge_split.py`).
- **Ổ đĩa cục bộ chỉ còn ~8 GB**: tải từng loại descriptor rồi xoá ngay sau khi tính metric.
- **Không chép số bằng tay vào báo cáo.** Lần đầu viết bảng BoQ bằng tay đã sai 7 ô. Mọi bảng giờ sinh bằng `vprspace_repro/report_tables.py` và `compare.py`.
- `pip freeze` của venv `--without-pip` từng **rỗng**: lệnh `venv/bin/python -m pip` không có pip. Đã sửa bootstrap để dùng `pip --python <venv>`. Phiên bản thật của các phiên cũ lấy từ dòng `torch 2.6.0+cu124` trong log worker.

## 8. DenseVLAD: từ kornia sang VLFeat

| Khác biệt | kornia `DenseSIFTDescriptor` | VLFeat `vl_phow` (MATLAB) |
|---|---|---|
| Làm mịn trước | $\sigma = \sqrt{(b/6)^2 - 0{,}25}$ (chúng tôi tự đặt, **sai**) | $\sigma = b/6$ (`vl_phow.m` 0.9.21) |
| Gom không gian | tam giác rộng b, không cửa sổ Gauss | nội suy song tuyến (2b−1), cửa sổ Gauss 1,5 bin ("fast": phẳng theo bin) |
| Ngưỡng tương phản | không có, nhiễu bị chuẩn hoá thành vector đơn vị | descriptor có chuẩn < 0,005 bị đưa về 0 |
| Lượng tử | float | uint8 = min(512·d, 255) |
| Bố cục 128 chiều | angle-major | MEX chuyển vị ảnh, rồi `vl_dsift_transpose_descriptor` |

Cách làm:

- `vprspace_repro/vlfeat_phow/phow_shim.c` gọi `vl_imsmooth` và `vl_dsift` **giống hệt** hai file MEX: ảnh column-major, bounds, chuyển vị, chuẩn uint8.
- Thư viện build trong notebook CPU `vprspace-vlfeat-build`: tải bản phát hành 0.9.21, kiểm SHA-256 `91a9dd85…`, rồi chạy self-test.
- **Không chạy được binary ngoài trong container của phiên Claude**: bộ phân loại quyền chặn. Mọi chạy VLFeat diễn ra trên Kaggle. Phần thuần Python

(imresize MATLAB, rgb2gray, RootSIFT, VLAD, whitening) được kiểm thử cục bộ bằng một `phow` giả lập.
- Trên Kaggle, thư viện `.so` được copy sang `/tmp` trước khi `dlopen`, vì input có thể mount `noexec`. Thư viện cũng được nạp trong tiến trình cha trước khi fork pool.

## 9. Mẹo vận hành Kaggle mới

- **Dừng một kernel treo**: push một version mới (code rỗng) cho cùng slug. Version đang chạy bị thay và trạng thái về COMPLETE ngay. CLI không có lệnh cancel.
- Internet Archive: API `archive.org/wayback/available` truy cập được từ container, nhưng `web.archive.org` bị chặn. Phải đọc snapshot qua notebook CPU có Internet.
- Trang lưu ở chế độ `id_` có thể trả nội dung **gzip**; cần giải nén trước khi parse.
- Dataset công khai lớn (GSV-Cities, 23 GB) gắn thẳng vào phiên GPU qua `dataset_sources`, không tốn thời gian tải. Plan đã hỗ trợ các trường `dataset_sources`, `extra_sources`, `pre` và `post`.